

Macros

Macros give Clojure great power. With most programming techniques, you build features *within* the language. When you write macros, it is more accurate to say that you are “adding features to” the language. This is a powerful and dangerous ability, so you should follow the rules in [Section 7.1, *When to Use Macros*, on page 165](#), at least until you have enough experience to decide for yourself when to bend the rules. [Section 7.2, *Writing a Control Flow Macro*, on page 166](#) jump-starts that experience, walking you through adding a new feature to Clojure.

While powerful, macros are not always simple. Clojure works to make macros as simple as is feasible by including conveniences to solve many common problems that occur when writing macros. [Section 7.3, *Making Macros Simpler*, on page 172](#) explains these problems and shows how Clojure mitigates them.

Macros are so different from other programming idioms that you may struggle to know when to use them. There is no better guide than the shared experience of the community, so [Section 7.4, *Taxonomy of Macros*, on page 177](#) introduces a taxonomy of Clojure macros, based on the macros in Clojure and contrib libraries.

7.1 When to Use Macros

Macro Club has two rules, plus one exception.

The first rule of Macro Club is Don’t Write Macros. Macros are complex, and they require you to think carefully about the interplay of macro expansion time and compile time. If you can write it as a function, think twice before using a macro.

The second rule of Macro Club is Write Macros If That Is the Only Way to Encapsulate a Pattern. All programming languages provide some way to encapsulate patterns, but without macros these mechanisms are incomplete. In most languages, you sense that incompleteness whenever you say, “My life would be easier if only my language had feature X.” In Clojure, you just implement feature X using a macro.

The exception to the rule is that *you can write any macro that makes life easier for your callers when compared with an equivalent function*. But to understand this exception, you need some practice writing macros and comparing them to functions. So, let’s get started with an example.

7.2 Writing a Control Flow Macro

Clojure provides the `if` special form as part of the language:

```
(if (= 1 1) (println "yep, math still works today"))
| yep, math still works today
```

Some languages have an `unless`, which is (almost) the opposite of `if`. `unless` performs a test and then executes its body only if the test is logically false.

Clojure doesn’t have `unless`, but it does have an equivalent macro called `when-not`. For the sake of having a simple example to start with, let’s pretend that `when-not` doesn’t exist and create an implementation of `unless`. To follow the rules of Macro Club, begin by trying to write `unless` as a function:

```
src/examples/macros.clj
; This is doomed to fail...
(defn unless [expr form]
  (if expr nil form))
```

Check that `unless` correctly evaluates its form when its test `expr` is false:

```
(unless false (println "this should print"))
| this should print
```

Things appear fine so far. But let’s be diligent and test the true case too:

```
(unless true (println "this should not print"))
| this should not print
```

Clearly something has gone wrong. The problem is that Clojure evaluates all the arguments before passing them to a function, so the `println` is called before `unless` *ever sees it*. In fact, both calls to `unless` earlier call `println` too soon, before entering the `unless` function. To see this, add a `println` inside `unless`:

```
(defn unless [expr form]
  (println "About to test...")
  (if expr nil form))
```

Now you can clearly see that function arguments are always evaluated before passing them to `unless`:

```
(unless false (println "this should print"))
| this should print
| About to test...

(unless true (println "this should not print"))
| this should not print
| About to test...
```

Macros solve this problem, because they do not evaluate their arguments immediately. Instead, you get to choose when (and if!) the arguments to a macro are evaluated.

When Clojure encounters a macro, it processes it in two steps. First, it expands (executes) the macro and substitutes the result back into the program. This is called *macro expansion time*. Then, it continues with the normal *compile time*.

To write `unless`, you need to write Clojure code to perform the following translation at macro expansion time:

```
(unless expr form) -> (if expr nil form)
```

Then, you need to tell Clojure that your code is a macro by using `defmacro`, which looks almost like `defn`:

```
(defmacro name doc-string? attr-map? [params*] body)
```

Because Clojure code is just Clojure data, you already have all the tools you need to write `unless`. Write the `unless` macro using `list` to build the `if` expression:

```
(defmacro unless [expr form]
  (list 'if expr nil form))
```

The body of `unless` executes at macro expansion time, producing an `if` form for compilation. If you enter this expression at the REPL:

```
(unless false (println "this should print"))
```

then Clojure will (invisibly to you) expand the `unless` form into the following:

```
(if false nil (println "this should print"))
```

Then, Clojure compiles and executes the expanded if form. Verify that unless works correctly for both true and false:

```
(unless false (println "this should print"))
| this should print
-> nil

(unless true (println "this should not print"))
-> nil
```

Congratulations, you have written your first macro. `unless` may seem pretty simple, but consider this: what you have just done is *impossible* in most languages. In languages without macros, special forms get in the way.

Special Forms, Design Patterns, and Macros

Clojure has no special syntax for code. Code is composed of data structures. This is true for normal functions but also for special forms and macros.

Consider a language with more syntactic variety, such as Java.¹ In Java, the most flexible mechanism for writing code is the instance method. Imagine that you are writing a Java program. If you discover a recurring pattern in some instance methods, you have the entire Java language at your disposal to encapsulate that recurring pattern.

Good so far. But Java also has lots of “special forms” (although they are not normally called by that name). Unlike Clojure special forms, which are just Clojure data, each Java special form has its own syntax. For example, `if` is a special form in Java. If you discover a recurring pattern of usage involving `if`, there is *no way to encapsulate* that pattern. You cannot create an `unless`, so you are stuck simulating `unless` with an idiomatic usage of `if`:

```
if (!something) ...
```

This may seem like a relatively minor problem. Java programmers can certainly learn to mentally make the translation from `if (!foo)` to `unless (foo)`. But the problem is not just with `if`: *every distinct syntactic form* in the language inhibits your ability to encapsulate recurring patterns involving that form.

As another example, Java `new` is a special form. Polymorphism is not available for `new`, so you must simulate polymorphism, for example with an idiomatic usage of a class method:

```
Widget w = WidgetFactory.makeWidget(...)
```

1. We are not trying to beat up on Java in particular; it is just easier to talk about a specific language, and Java is well known.

This idiom is a little bulkier. It introduces a whole new class, `WidgetFactory`. This class is meaningless in the problem domain and exists only to work around the constructor special form. Unlike the `unless` idiom, the “polymorphic instantiation” idiom is complicated enough that there is more than one way to implement a solution. Thus, the idiom should more properly be called a *design pattern*.

Wikipedia defines a design pattern² to be a “general reusable solution to a commonly occurring problem in software design.” It goes on to state that a “design pattern is not a finished design that can be transformed *directly* (emphasis added) into code.”

That is where macros fit in. Macros provide a layer of *indirection* so that you can *automate the common parts of any recurring pattern*. Macros and code-as-data work together, enabling you to reprogram your language on the fly to encapsulate patterns.

Of course, this argument does not go entirely in one direction. Many people would argue that having a bunch of special syntactic forms makes a programming language easier to learn or read. We do not agree, but even if we did, we would be willing to trade syntactic variety for a powerful macro system. Once you get used to code as data, the ability to automate design patterns is a huge payoff.

Expanding Macros

When you created the `unless` macro, you quoted the symbol `if`:

```
(defmacro unless [expr form]
  (list 'if expr nil form))
```

But you did not quote any other symbols. To understand why, you need to think carefully about what happens at macro expansion time:

- By quoting `'if`, you prevent Clojure from directly evaluating `if` at macro expansion time. Instead, evaluation strips off the quote, leaving `if` to be compiled.
- You do not want to quote `expr` and `form`, because they are macro arguments. Clojure will substitute them without evaluation at macro expansion time.
- You do not need to quote `nil`, since `nil` evaluates to itself.

2. [http://en.wikipedia.org/wiki/Design_pattern_\(computer_science\)](http://en.wikipedia.org/wiki/Design_pattern_(computer_science))

Thinking about what needs to be quoted can get complicated quickly. Fortunately, you do not have to do this work in your head. Clojure includes diagnostic functions so that you can test macro expansions at the REPL.

The function `macroexpand-1` will show you what happens at macro expansion time:

```
(macroexpand-1 form)
```

Use `macroexpand-1` to prove that `unless` expands to a sensible `if` expression:

```
(macroexpand-1 '(unless false (println "this should print")))
-> (if false nil (println "this should print"))
```

Macros are complicated beasts, and we cannot overstate the importance of testing them with `macroexpand-1`. Let's go back and try some incorrect versions of `unless`. Here is one that incorrectly quotes the `expr`:

```
(defmacro bad-unless [expr form]
  (list 'if 'expr nil form))
```

When you expand `bad-unless`, you will see that it generates the symbol `expr`, instead of the actual test expression:

```
(macroexpand-1 '(bad-unless false (println "this should print")))
-> (if expr nil (println "this should print"))
```

If you try to actually use the `bad-unless` macro, Clojure will complain that it cannot resolve the symbol `expr`:

```
(bad-unless false (println "this should print"))
-> java.lang.Exception: Unable to resolve symbol: expr in this context
```

Sometimes macros expand into other macros. When this happens, Clojure will continue to expand all macros, until only normal code remains. For example, the `..` macro expands recursively, producing a dot operator call, wrapped in another `..` to handle any arguments that remain. You can see this with the following macro expansion:

```
(macroexpand-1 '(.. arm getHand getFinger))
-> (clojure.core/.. (. arm getHand) getFinger)
```

If you want to see `..` expanded all the way, use `macroexpand`:

```
(macroexpand form)
```

If you `macroexpand` a call to `..`, it will recursively expand until only dot operators remain:

```
(macroexpand '(.. arm getHand getFinger))
-> (. (. arm getHand) getFinger)
```

(It is not a problem that `arm`, `getHand`, and `getFinger` do not exist. You are only expanding them, not attempting to compile and execute them.)

Another recursive macro is `and`. If you call `and` with more than two arguments, it will expand to include another call to `and`, with one less argument:

```
(macroexpand '(and 1 2 3))
-> (let* [and__3585__auto__ 1]
      (if and__3585__auto__ (clojure.core/and 2 3)
        and__3585__auto__))
```

This time, `macroexpand` does *not* expand all the way. `macroexpand` works only against the top level of the form you give it. Since the expansion of `and` creates a new `and` nested inside the form, `macroexpand` does not expand it.

when and when-not

Your `unless` macro could be improved slightly to execute multiple forms, avoiding this error:

```
(unless false (println "this") (println "and also this"))
-> java.lang.IllegalArgumentException: \
Wrong number of args passed to: macros$unless
```

Think about how you would write the improved `unless`. You would need to capture a variable argument list and stick a `do` in front of it so that every form executes. Clojure provides exactly this behavior in its `when` and `when-not` macros:

```
(when test & body)
```

```
(when-not test & body)
```

`when-not` is the improved `unless` you are looking for:

```
(when-not false (println "this") (println "and also this"))
| this
| and also this
-> nil
```

Given your practice writing `unless`, you should now have no trouble reading the source for `when-not`:

```
; from Clojure core
(defmacro when-not [test & body]
  (list 'if test nil (cons 'do body)))
```

And, of course, you can use `macroexpand-1` to see how `when-not` works:

```
(macroexpand-1 '(when-not false (print "1") (print "2")))
-> (if false nil (do (print "1") (print "2")))
```

when is the opposite of when-not and executes its forms only when its test is true. Note that when differs from if in two ways:

- if allows an else clause, and when does not. This reflects English usage, because nobody says “when ... else.”
- Since when does not have to use its second argument as an else clause, it is free to take a variable argument list and execute all the arguments inside a do.

You don’t really need an unless macro. Just use Clojure’s when-not. Always check to see whether somebody else has written the macro you need.

7.3 Making Macros Simpler

The unless macro is a great simple example, but most macros are more complex. In this section, we will build a set of increasingly complex macros, introducing Clojure features as we go. For your reference, the features introduced in this section are summarized in the following table.

Form	Description
foo#	Auto-gensym: Inside a syntax-quoted section, create a unique name prefixed with foo.
(gensym prefix?)	Create a unique name, with optional prefix.
(macroexpand form)	Expand form with macroexpand-1 repeatedly until the returned form is no longer a macro.
(macroexpand-1 form)	Show how Clojure will expand form.
(list-frag? ~@form list-frag?)	Splicing unquote: Use inside a syntax quote to splice an unquoted list into a template.
`form	Syntax quote: Quote form, but allow internal unquoting so that form acts a template. Symbols inside form are resolved to help prevent inadvertent symbol capture.
~form	Unquote: Use inside a syntax quote to substitute an unquoted value.

First let’s build a replica of Clojure’s .. macro. We’ll call it chain, since it chains a series of method calls. Here are some sample expansions of chain:

Macro Call	Expansion
(chain arm getHand)	(. arm getHand)
(chain arm getHand getFinger)	(. (. arm getHand) getFinger)

Begin by implementing the simple case where the chain calls only one method. The macro needs only to make a simple list:

```
src/examples/macros/chain_1.clj
; chain reimplements Clojure's .. macro
(defmacro chain [x form]
  (list '. x form))
```

chain needs to support any number of arguments, so the rest of the implementation should define a recursion. The list manipulation becomes more complex, since you need to build two lists and concat them together:

```
src/examples/macros/chain_2.clj
(defmacro chain
  ([x form] (list '. x form))
  ([x form & more] (concat (list 'chain (list '. x form)) more)))
```

Test chain using macroexpand to make sure it generates the correct expansions:

```
(macroexpand '(chain arm getHand))
-> (. arm getHand)

(macroexpand '(chain arm getHand getFinger))
-> (. (. arm getHand) getFinger)
```

The chain macro works fine as written, but it is difficult to read the expression that handles more than one argument:

```
(concat (list 'chain (list '. x form)) more)))
```

The definition of chain oscillates between macro code and the body to be generated. The intermingling of the two makes the entire thing hard to read. And this is just a baby of a form, only one line in length. As macro forms grow more complex, assembly functions such as list and concat quickly obscure the meaning of the macro.

One solution to this kind of problem is a templating language. If macros were created from templates, you could take a “fill-in-the-blanks” approach to creating them. The definition of chain might look like this:

```
; hypothetical templating language
(defmacro chain
  ([x form] (. ${x} ${form}))
  ([x form & more] (chain (. ${x} ${form}) ${more})))
```

In this hypothetical templating language, the `${}` lets you substitute arguments into the macro expansion.

Notice how much easier the definition is to read and how it clearly shows what the expansion will look like.

Syntax Quote, Unquote, and Splicing Unquote

Clojure macros support templating without introducing a separate language. The *syntax quote* character, which is a backquote (`), works almost like normal quoting. But inside a syntax-quoted list, the *unquote character* (~, a tilde) turns quoting off again. The overall effect is templates that look like this:

```
src/examples/macros/chain_3.clj
(defmacro chain [x form]
  `( . ~x ~form))
```

Test that this new version of chain can correctly generate a single method call:

```
(macroexpand '(chain arm getHand))
-> (. arm getHand)
```

Unfortunately, the syntax quote/unquote approach will not quite work for the multiple-argument variant of chain:

```
src/examples/macros/chain_4.clj
; Does not quite work
(defmacro chain
  ([x form] `( . ~x ~form))
  ([x form & more] `(chain (. ~x ~form) ~more)))
```

When you expand this chain, the parentheses aren't quite right:

```
(macroexpand '(chain arm getHand getFinger))
-> (. (. arm getHand) (getFinger))
```

The last argument to chain is a list of more arguments. When you drop more into the macro “template,” it has parentheses because it is a list. But you don't want these parentheses; you want more to be *spliced* into the list. This comes up often enough that there is a reader macro for it: *splicing unquote* (~@). Rewrite chain using splicing unquote to splice in more:

```
src/examples/macros/chain_5.clj
(defmacro chain
  ([x form] `( . ~x ~form))
  ([x form & more] `(chain (. ~x ~form) ~@more)))
```

Now, the expansion should be spot on:

```
(macroexpand '(chain arm getHand getFinger))
-> (. (. arm getHand) getFinger)
```

Many macros follow the pattern of chain, aka Clojure ..:

1. Begin the macro body with a syntax quote (`) to treat the entire thing as a template.

2. Insert individual arguments with an unquote (~).
3. Splice in more arguments with splicing unquote (~@).

The macros we have built so far have been simple enough to avoid creating any bindings with `let` or binding. Let's create such a macro next.

Creating Names in a Macro

Clojure has a time macro that times an expression, writing the elapsed time to the console:

```
(time (str "a" "b"))
| "Elapsed time: 0.06 msecs"
-> "ab"
```

Let's build a variant of time called `bench`, designed to collect data across many runs. Instead of writing to the console, `bench` will return a map that includes both the return value of the original expression and the elapsed time.

The best way to begin writing a macro is to write its desired expansion by hand. `bench` should expand like this:

```
; (bench (str "a" "b"))
; should expand to
(let [start (System/nanoTime)
      result (str "a" "b")]
  {:result result :elapsed (- (System/nanoTime) start)})

-> {:elapsed 61000, :result "ab"}
```

The `let` binds `start` to the start time and then executes the expression to be benched, binding it to `result`. Finally, the form returns a map including the result and the elapsed time since `start`.

With the expansion in hand, you can now work backward and write the macro to generate the expansion. Using the technique from the previous section, try writing `bench` using syntax quoting and unquoting:

```
src/examples/macros/bench_1.clj
; This won't work
(defmacro bench [expr]
  `(let [start (System/nanoTime)
        result ~expr]
     {:result result :elapsed (- (System/nanoTime) start)}))
```

If you try to call this version of `bench`, Clojure will complain:

```
(bench (str "a" "b"))
-> java.lang.Exception: Can't let qualified name: examples.macros/start
```

Clojure is accusing you of trying to let a qualified name, which is illegal. Calling `macroexpand-1` confirms the problem:

```
(macroexpand-1 '(bench (str "a" "b")))
-> (clojure.core/let [examples.macros/start (System/nanoTime)
                    examples.macros/result (str "a" "b")]
    {:elapsed (clojure.core/- (System/nanoTime) examples.macros/start)
     :result examples.macros/result})
```

When a syntax-quoted form encounters a symbol, it resolves the symbol to a fully qualified name. At the moment, this seems like an irritant, because you *want* to create local names, specifically `start` and `result`. But Clojure's approach protects you from a nasty macro bug called *symbol capture*.

What would happen if macro expansion *did* allow the unqualified symbols `start` and `result` and then `bench` was later used in a scope where those names were already bound to something else? The macro would *capture* the names and bind them to different values, with bizarre results. If `bench` captured its symbols, it would appear to work fine most of the time. Adding one and two would give you three:

```
(let [a 1 b 2]
  (bench (+ a b)))

-> {:result 3, :elapsed 39000}
```

...until the unlucky day that you picked a local name like `start`, which collided with a name inside `bench`:

```
(let [start 1 end 2]
  (bench (+ start end)))

-> {:result 1228277342451783002, :elapsed 39000}
```

`bench` captures the symbol `start` and binds it to `(System/nanoTime)`. All of a sudden, one plus two seems to equal 1228277342451783002.

Clojure's insistence on resolving names in macros helps protect you from symbol capture, but you still don't have a working `bench`. You need some way to introduce local names, ideally *unique* ones that cannot collide with any names used by the caller.

Clojure provides a reader form for creating unique local names. Inside a syntax-quoted form, you can append an octothorpe (`#`) to an unqualified name, and Clojure will create an autogenerated symbol, or *auto-gensym*: a symbol based on the name plus an underscore and a unique ID. Try it at the REPL:

```
`foo#
foo__1004
```

With automatically generated symbols at your disposal, it is easy to implement bench correctly:

```
(defmacro bench [expr]
  `(let [start# (System/nanoTime)
        result# ~expr]
    {:result result# :elapsed (- (System/nanoTime) start#)}))
```

Test it at the REPL:

```
(bench (str "a" "b"))
-> {:elapsed 63000, :result "ab"}
```

Clojure makes it easy to generate unique names, but if you are determined, you can still force symbol capture. The sample code for the book includes an evil-bench that shows a combination of syntax quoting, quoting, and unquoting that leads to symbol capture. Don't use symbol capture unless you have a thorough understanding of macros.

7.4 Taxonomy of Macros

Now that you have written several macros, we can restate the rules of Macro Club with more supporting detail.

The first rule of Macro Club is Don't Write Macros. Macros are complex. If none of the macros in Clojure seems complex to you, my company is hiring.³

The second rule of Macro Club is Write Macros If That Is the Only Way to Encapsulate a Pattern. As you have seen, the patterns that resist encapsulation tend to arise around special forms, which are irregularities in a language. So, the second rule can also be called the Special Form Rule.

Special forms have special powers that you, the programmer, do not have:

- Special forms provide the most basic flow control structures, such as if and recur. All flow control macros must eventually call a special form.
- Special forms provide direct access to Java. Whenever you call Java from Clojure, you are going through at least one special form, such as the dot or new.
- Names are created and bound through special forms, whether defining a var with def, creating a lexical binding with let, or creating a dynamic binding with binding.

3. <http://thinkrelevance.com>

As powerful as they are, special forms are not functions. They cannot do some things that functions can do. You cannot apply a special form, store a special form in a var, or use a special form as a filter with the sequence library. In short, special forms are not first-class citizens of the language.

The specialness of special forms could be a major problem and lead to repetitive, unmaintainable patterns in your code. But macros neatly solve the problem, because you can use macros to generate special forms. In a practical sense, *all language features are first-class features at macro expansion time*.

Macros that generate special forms are often the most difficult to write but also the most rewarding. As if by magic, such macros seem to *add new features to the language*.

The exception to the Macro Club rules is caller convenience: *you can write any macro that makes life easier for your callers when compared with an equivalent function*. Because macros do not evaluate their arguments, callers can pass raw code to a macro, instead of wrapping the code in an anonymous function. Or, callers can pass unescaped names, instead of quoted symbols or strings.

We have reviewed the macros in Clojure and contrib libraries, and almost all of them follow the rules of Macro Club. Also, they fit into one or more of the categories shown in the following table, which shows the taxonomy of Clojure macros.

Justification	Category	Examples
Special form	Conditional evaluation	when, when-not, and, or, comment
Special form	Defining vars	defn, defmacro, defmulti, defstruct, declare
Special form	Java interop	..., doto, import-static
Caller convenience	Postponing evaluation	lazy-cat, lazy-seq, delay
Caller convenience	Wrapping evaluation	with-open, dosync, with-out-str, time, assert
Caller convenience	Avoiding a lambda	(Same as for “Wrapping evaluation”)

Let’s examine each of the categories in turn.

Conditional Evaluation

Because macros do not immediately evaluate their arguments, they can be used to create custom control structures. You have already seen this with the unless example in [Section 7.2, Writing a Control Flow Macro, on page 166](#).

Macros that do conditional evaluation tend to be fairly simple to read and write. They follow a common form: evaluate some argument (the condition); then, based on that evaluation, pick which other arguments to evaluate, if any. A good example is Clojure's `and`:

```
Line 1 (defmacro and
2   ([] true)
3   ([x] x)
4   ([x & rest]
5     `(let [and# ~x]
6       (if and# (and ~@rest) and#))))
```

`and` is defined recursively. The zero- and one-argument bodies set up the base cases:

- For no arguments, return `true`.
- For one argument, return that argument.

For two or more arguments, `and` uses the first argument as its condition, evaluating it on line 5. Then, if the condition is true, `and` proceeds to evaluate the remaining arguments by recursively `anding` the rest (line 6).

`and` must be a macro in order to short-circuit evaluation after the first nontrue value is encountered. Unsurprisingly, `and` has a close cousin macro, `or`. Their signatures are the same:

```
(and & exprs)
```

```
(or & exprs)
```

The difference is that `and` stops on the first logical false, while `or` stops on the first logical true:

```
(and 1 0 nil false)
-> nil
```

```
(or 1 0 nil false)
-> 1
```

The all-time short-circuit evaluation champion is the `comment` macro:

```
(comment & exprs)
```

`comment` never evaluates *any* of its arguments and is sometimes used at the end of a source code file to demonstrate the usage of an API.

For example, the Clojure inspector library ends with the following `comment`, demonstrating the use of the inspector:

```
(comment

(load-file "src/inspector.clj")
(refer 'inspector)
(inspect-tree {:a 1 :b 2 :c [1 2 3 {:d 4 :e 5 :f [6 7 8]}]})
(inspect-table [[1 2 3][4 5 6][7 8 9][10 11 12]])

)
```

Notice the lack of indentation. This would be nonstandard in most Clojure code but is useful in comment, whose purpose is to draw attention to its body.

Creating Vars

Clojure vars are created by the `def` special form. Anything else that creates a var must eventually call `def`. So, for example, `defn`, `defmacro`, and `defmulti` are all themselves macros.

To demonstrate writing macros that create vars, we will look at two macros that are also part of Clojure: `defstruct` and `declare`.

Clojure provides a low-level function for creating structs called `create-struct`:

```
(create-struct & key-symbols)
```

Use `create-struct` to create a person struct:

```
(def person (create-struct :first-name :last-name))
-> #'user/person
```

`create-struct` works, but it is visually noisy. Given that you often want to immediately `def` a new struct, you will typically call `defstruct`, which combines `def` and `create-struct` in a single operation:

```
(defstruct name & key-symbols)
```

`defstruct` is a simple macro, and it is already part of Clojure:

```
(defmacro defstruct
  [name & keys]
  `(def ~name (create-struct ~@keys)))
```

This macro looks so simple that you may be tempted to try to write it as a function. You won't be able to, because `def` is a special form. You must generate `def` at macro time; you cannot make “dynamic” calls to `def` at runtime.

`defstruct` makes a single line easier to read, but some macros can also condense many lines down into a single form. Consider the problem of forward declarations. You are writing a program that needs forward references to vars `a`, `b`,

c, and d. You can call `def` with no arguments to define the var names without an initial binding:

```
(def a)
(def b)
(def c)
(def d)
```

But this is tedious and wastes a lot of vertical space. The `declare` macro takes a variable list of names and `defs` each name for you:

```
(declare & names)
```

Now you can declare all the names in a single compact form:

```
(declare a b c d)
-> #'user/d
```

The implementation of `declare` is built into Clojure:

```
(defmacro declare
  [& names] `(do ~@(map #(list 'def %) names)))
```

Let's analyze `declare` from the inside out. The anonymous function `#(list 'def %)` is responsible for generating a single `def`. Test this form alone at the REPL:

```
(#(list 'def %) 'a)
-> (def a)
```

The `map` invokes the inner function once for each symbol passed in. Again, you can test this form at the REPL:

```
(map #(list 'def %) '[a b c d])
-> ((def a) (def b) (def c) (def d))
```

Finally, the leading `do` makes the entire expansion into a single legal Clojure form:

```
`(do ~@(map #(list 'def %) '[a b c d]))
-> (do (def a) (def b) (def c) (def d))
```

Substituting `'[a b c d]` in the previous form is the manual equivalent of testing the entire macro with `macroexpand-1`:

```
(macroexpand-1 '(declare a b c d))
-> (do (def a) (def b) (def c) (def d))
```

Many of the most interesting parts of Clojure are macros that expand into special forms involving `def`. We have explored a few here, but you can read the source of any of them. Most of them live at `src/clj/clojure/core.clj` in the Clojure source distribution.

Java Interop

Clojure programs call into Java via the `.` (dot), `new`, and `set!` special forms. However, idiomatic Clojure code often uses macros such as `..` (threaded member access) and `doto` to simplify forms that call Java.

You (or anyone else) can extend how Clojure calls Java by writing a macro. Consider the following scenario. You are writing code that uses several of the constants in `java.lang.Math`:

```
Math/PI
-> 3.141592653589793
(Math/pow 10 3)
-> 1000.0
```

In a longer segment of code, the `Math/` prefix would quickly become distracting, so it would be nice if you could say simply `PI` and `pow`. Clojure doesn't provide any direct way to do this, but you could define a bunch of vars by hand:

```
(def PI Math/PI)
-> #'user/PI
(defn pow [b e] (Math/pow b e))
-> #'user/pow
```

Stuart Sierra automated the boilerplate with the `import-static` macro:

```
(examples.import-static/import-static class & members)
```

`import-static` imports static members of a Java class as names in the local namespace. Use `import-static` to import the members you want from `Math`.

```
(use '[examples.import-static :only (import-static)])
(import-static java.lang.Math PI pow)
-> nil
```

```
PI
-> 3.141592653589793
```

```
(pow 10 3)
-> 1000.0
```

Postponing Evaluation

Most sequences in Clojure are lazy. When you are building a lazy sequence, you often want to combine several forms whose evaluation is postponed until the sequence is forced. Since evaluation is not immediate, a macro is required.

You have already seen such a macro in [Section 3.3, *Lazy and Infinite Sequences*, on page 69](#): `lazy-seq`. Another example is `delay`:

```
(delay & exprs)
```

When you create a delay, it holds on to its `exprs` and does nothing with them until it is forced to. Try creating a delay that simulates a long calculation by sleeping:

```
(def slow-calc (delay (Thread/sleep 5000) "done!"))
-> #'user/slow-calc
```

To actually execute the delay, you must force it:

```
(force x)
```

Try forcing your `slow-calc` a few times:

```
(force slow-calc)
-> "done!"
(force slow-calc)
-> "done!"
```

The first time you force a delay, it executes its expressions and caches the result. Subsequent forces simply return the cached value.

The macros that implement lazy and delayed evaluation all call Java code in `clojure.jar`. In your own code, you should not call such Java APIs directly. Treat the lazy/delayed evaluation macros as the public API, and treat the Java classes as implementation detail that is subject to change.

Wrapping Evaluation

Many macros wrap the evaluation of a set of forms, adding some special semantics before and/or after the forms are evaluated. You have already seen several examples of this kind of macro:

- `time` starts a timer, evaluates forms, and then reports how long they took to execute.
- `let` and binding establish bindings, evaluate some forms, and then tear down the bindings.
- `with-open` takes an open file (or other resource), executes some forms, and then makes sure the resource is closed in a `finally` block.
- `dosync` executes forms within a transaction.

Another example of a wrapper macro is `with-out-str`:

```
(with-out-str & exprs)
```

`with-out-str` temporarily binds `*out*` to a new `StringWriter`, evaluates its `exprs`, and then returns the string written to `*out*`. `with-out-str` makes it easy to use `print` and `println` to build strings on the fly:

```
(with-out-str (print "hello, ") (print "world"))
-> "hello, world"
```

The implementation of `with-out-str` has a simple structure that can act as a template for writing similar macros:

```
Line 1 (defmacro with-out-str
2   [& body]
3   `(let [s# (new java.io.StringWriter)]
4       (binding [*out* s#]
5         ~@body
6         (str s#))))
```

Wrapper macros usually take a variable number of arguments (line 2), which are the forms to be evaluated. They then proceed in three steps:

1. *Setup*: Create some special context for evaluation, introducing bindings with `let` (line 3) and bindings (line 4) as necessary.
2. *Evaluation*: Evaluate the forms (line 5). Since there are typically a variable number of forms, insert them via a splicing unquote: `~@`.
3. *Teardown*: Reset the execution context to normal and return a value as appropriate (line 6)

When writing a wrapper macro, always ask yourself whether you need a `finally` block to implement the teardown step correctly. For `with-out-str`, the answer is no, because both `let` and `binding` take care of their own cleanup. If, however, you are setting some global or thread-local state via a Java API, you will need a `finally` block to reset this state.

This talk of mutable state leads to another observation. Any code whose behavior changes when executed inside a wrapper macro is obviously *not* a pure function. `print` and `println` behave differently based on the value of `*out*` and so are not pure functions. Macros that set a binding, such as `with-out-str`, do so to alter the behavior of an impure function somewhere.

Not all wrappers change the behavior of the functions they wrap. You've already seen `time`, which times a function's execution. Another example is `assert`:

```
(assert expr)
```

`assert` tests an expression and raises an exception if it is not logically true:

```
(assert (= 1 1))
-> nil
```

```
(assert (= 1 2))
-> java.lang.Exception: Assert failed: (= 1 2)
```

Macros like `assert` and `time` violate the first rule of Macro Club in order to avoid unnecessary lambdas.

Avoiding Lambdas

For historical reasons, anonymous functions are often called *lambdas*. Sometimes a macro can be replaced by a function call, with the arguments wrapped in a lambda. For example, the `bench` macro from [Syntax Quote, Unquote, and Splicing Unquote, on page 174](#) does not need to be a macro. You can write it as a function:

```
(defn bench-fn [f]
  (let [start (System/nanoTime)
        result (f)]
    {:result result :elapsed (- (System/nanoTime) start)}))
```

However, if you want to call `bench-fn`, you must pass it a function that wraps the form you want to execute. The following code shows the difference:

```
; macro
(bench (+ 1 2))
-> {:elapsed 44000, :result 3}

; function
(bench-fn (fn [] (+ 1 2)))
-> {:elapsed 53000, :result 3}
```

For things like `bench`, macros and anonymous functions are near substitutes. Both prevent immediate execution of a form. However, the anonymous function approach requires more work on the part of the caller, so it is OK to break the first rule and write a macro instead of a function.

Another reason to prefer a macro for `bench` is that `bench-fn` is not a perfect substitute; it adds the overhead of an anonymous function call at runtime. Since `bench`'s purpose is to time things, you should avoid this overhead.

7.5 Wrapping Up

Clojure macros let you automate patterns in your code. Because they transform source code at macro expansion time, you can use macros to abstract away *any* kind of pattern in your code. You are not limited to working within Clojure. With macros, you can *extend* Clojure into your problem domain.